

SEGMENT TREE

```
int a[100005], seg[4*100005], lazy[4*100005];
```

```
void build(int idx, int low, int high){
    if(low == high){
        seg[idx] = a[low];
        return;
    }

    int mid = (low+high)/2;
    build(2*idx+1, low, mid);
    build(2*idx+2, mid+1, high);
    seg[idx] = max(seg[2*idx+1], seg[2*idx+2]);
    seg[idx] = gcd(seg[2*idx+1], seg[2*idx+2]);
}

int query(int idx, int low, int high, int l, int r){
    if(low >= l && high <= r){
        return seg[idx];
    }
    if(high < l || low > r) return INT_MIN;
    int mid = (low+high)/2;
    int left = query(2*idx+1, low, mid, l, r);
    int right = query(2*idx+2, mid+1, high, l, r);
    return max(left, right);
}

void update(int idx, int low, int high, int pos, int val){
    if (pos < low || pos > high){
        return;
    }
    if (low == high){
        seg[idx] = val;
        return;
    }
    if(low >= pos && high <= pos){
        seg[idx] += val;
        return;
    }
    int mid = (low + high) / 2;
    int left = 2 * idx;
    int right = 2 * idx + 1;
    update(left, low, mid, pos, val);
    update(right, mid + 1, high, pos, val);
    // seg[idx] = seg[left] + seg[right];
    seg[idx] = min(seg[left], seg[right]);
}
```

```

int lazy_update(int idx, int low, int high, int pos, int Newvalue){
    if(lazy[idx] != 0){
        seg[idx] += lazy[idx];

        if(low != high){
            lazy[idx*2] += lazy[idx];
            lazy[idx * 2 + 1] += lazy[idx];
        }
        lazy[idx] = 0;
    }

    if (pos < low || pos > high){
        return;
    }

    if(low >= pos && high <= pos){
        seg[idx] += Newvalue;
        if(low != high){
            lazy[idx * 2] += lazy[idx];
            lazy[idx * 2 + 1] += lazy[idx];
        }
    }
    int left = idx*2;
    int right = idx*2+1;
    int mid = (low+high)/2;

    lazy_update(left,low,mid,pos,Newvalue);
    lazy_update(right,mid+1,high,pos,Newvalue);
    seg[idx] = max(seg[left],seg[right]);
    seg[idx] = min(seg[left],seg[right]);
}

- /
void printStatus(int n){
    int powerof2 = 1;
    while(powerof2 < n) powerof2*= 2;
    int usedNodes = 2*powerof2-1;

    for(int i=1; i<=usedNodes; i++){
        cout << seg[i] << " ";
    }
    cout << endl;

    for(int i=1; i<=usedNodes; i++){
        cout << lazy[i] << " ";
    }
    cout << endl;
}

const int INF = 1e9;
vector<int>arr,seg,lazy;

void push(int idx, int low, int high){
    if(lazy[idx] != 0){
        seg[idx] += lazy[idx];
        if(low != high){
            lazy[idx*2] += lazy[idx];
            lazy[idx*2+1] += lazy[idx];
        }
        lazy[idx] = 0;
    }
}

```

```

void update(int idx, int low, int high, int l, int r, int val){
    push(idx, low, high);

    if(low > r || high < l) return;

    if(low >= l && high <= r){
        lazy[idx] += val;
        push(idx, low, high);
        return;
    }

    int mid = (low+high)/2;
    int left = 2*idx ;
    int right = 2*idx+1;

    update(left, low, mid, l, r, val);
    update(right, mid+1, high, l, r, val);
    seg[idx] = min(seg[left], seg[right]);
}

```

RANGE QUERIES

```

//.....
string s;
if (!(cin >> s)) return 0;

int n = s.length();
// 2D Prefix Sum: pref[character_index][string_index]
vector<vector<int>> pref(26, vector<int>(n + 1, 0));

// Build the Prefix Sum table
for (int i = 0; i < n; i++) {
    int charIdx = s[i] - 'a';
    for (int c = 0; c < 26; c++) {
        pref[c][i + 1] = pref[c][i]; // Carry over previous counts
    }
    pref[charIdx][i + 1]++; // Increment count for current character
}

int q;
cin >> q;
while (q--) {
    int l, r;
    cin >> l >> r;

    int maxCount = -1;
    char bestChar = 'a';

    // Check all 26 characters for this range
    for (int c = 0; c < 26; c++) {
        int currentCount = pref[c][r] - pref[c][l - 1];

        if (currentCount > maxCount) {
            maxCount = currentCount;
            bestChar = (char)(c + 'a');
        }
    }

    cout << bestChar << ":" << maxCount << endl;
}

```

SPARSE TABLE

```
int query(int l, int r, vector<vector<int>>&sparse){
    int win = r-l+1;    //size of the query
    int j = log2(win);  //choosing the largest block size fitting in this range

    int left = sparse[l][j];    //first block
    int right = sparse[r-(1<<j)+1][j];    //second block

    int ans = min(left, right);
    cout << "Min=" << ans << " ";
    cout << "min([" << l << ", " << l + (1 << j) - 1 << "], ";
    cout << "[" << r - (1 << j) + 1 << ", " << r << "])";

    return ans;
}

int n;
cin >> n;

vector<int>arr(n);
for(int i=0; i<n; i++){
    cin >> arr[i];
}

int q;
cin >> q;

int k = log2(n)+1;
vector<vector<int>>sparse(n, vector<int>(k));

cout << "Ranges to be stored in Sparse Table:" << endl;

for(int i=0; i<n; i++){
    sparse[i][0] = arr[i];
    cout << '(' << i << ', ' << i << ')' ;
    for(int j=1; j<k; j++){
        if(i+ (1 << j) -1 < n){
            cout << " (" << i << ', ' << i + (1 << j) - 1 << ')' ;
        }
    }
    cout << endl;
}

for (int j = 1; j < k; j++)
{
    for (int i = 0; i + (1 << j) <= n; i++)
    {
        sparse[i][j] = min(
            sparse[i][j - 1],
            sparse[i + (1 << (j - 1))][j - 1]);
    }
}
```

```

cout << "\nStatus of Sparse Table:" << endl;

for (int i = 0; i < n; i++)
{
    for (int j = 0; j < k; j++)
    {
        if (i + (1 << j) <= n)
            cout << sparse[i][j] << " ";
    }
    cout << endl;
}

cout << endl;

for(int i = 1; i<= q; i++){
    int l,r;
    cin >> l >> r;

    cout << "Query- " << i << ": ";
    query(l, r, sparse);
    cout << endl;
}

// 1. Find the Least Significant Bit
int lsb(int i) {
    return i & (-i);
}

// 2. Linear Construction: Builds the tree in O(N)
void build(int n, const vector<long long>& arr, vector<long long>& tree) {
    for (int i = 1; i <= n; i++) {
        tree[i] = arr[i];
    }
    for (int i = 1; i <= n; i++) {
        int parent = i + lsb(i);
        if (parent <= n) {
            tree[parent] += tree[i];
        }
    }
}

// 3. Prefix Sum with Path Tracking
long long queryPrefix(int idx, const vector<long long>& tree, vector<int>& path) {
    long long sum = 0;
    while (idx > 0) {
        path.push_back(idx);
        sum += tree[idx];
        idx -= lsb(idx);
    }
    return sum;
}

```

vector<long long> arr(n + 1), vector<long long> tree(n + 1, 0);

```

// Output: Ranges stored in the table
cout << "Ranges to be stored in the Fenwick Table:" << endl;
for (int i = 1; i <= n; i++) {
    int start = i - lsb(i) + 1;
    cout << i << ":(" << start << "," << i << ")" << (i % 4 == 0 ? "\n" : " ");
}
cout << endl;

// Build and Output Status
build(n, arr, tree);
cout << "Status of Fenwick Tree (idx: value):" << endl;
for (int i = 1; i <= n; i++) {
    cout << i << ":" << tree[i] << (i == n ? "" : " ");
}
cout << "\n" << endl;

```

```

int q;
cin >> q;
for (int k = 1; k <= q; k++) {
    int L, R;
    cin >> L >> R;

    vector<int> pathR, pathL;
    long long sumR = queryPrefix(R, tree, pathR);
    long long sumL = queryPrefix(L - 1, tree, pathL);
    long long ans = sumR - sumL;

    // Output formatting to match sample
    cout << "Query-" << k << ": Sum=" << ans << " [Tree idx: sum(";
    for (int i = 0; i < pathR.size(); i++) {
        cout << pathR[i] << (i == pathR.size() - 1 ? "" : ",");
    }
    cout << ")";

    if (L > 1) {
        cout << " - sum(";
        for (int i = 0; i < pathL.size(); i++) {
            cout << pathL[i] << (i == pathL.size() - 1 ? "" : ",");
        }
        cout << ")";
    }
    cout << "]" << endl;
}

```

MINIMUM NUMBER OF BITS TO FLIP

```

int main() {
    int n, q;
    while (!(cin >> n >> q)) return 0;
    vector<int> arr(n);

    for (int i = 0; i < n; i++) {
        cin >> arr[i];
    }

    for (int i = 0; i < q; i++) {
        int l, r;
        cin >> l >> r;

        l = l - 1;
        r = r - 1;

        int len = r - l + 1;
        if ((len % 2) != 0) {
            cout << "-1" << endl;
        }
        else {
            int count_1 = 0;
            for (int i = l; i <= r; i++) {
                if (arr[i] == 1) {
                    count_1++;
                }
            }
            int target = len / 2;

            int flips = abs(count_1 - target);
            cout << flips << endl;
        }
    }
    return 0;
}

```

Each query checks whether the **bitwise OR** of a given range matches the **i-th security log entry**.

```

int n;
while(!(cin >> n)) return 0;
vector<int>arr(n);

for(int i=0; i<n; i++){
    cin >> arr[i];
}

int max_j = log2(n);
vector<vector<int>>>s(n, vector<int>(max_j+1));

for(int i=0; i<n; i++){
    s[i][0] = arr[i];
}

for(int j=1; j<=max_j; j++){
    for(int i=0; i+(1<<j)-1 <n; i++){
        s[i][j] = s[i][j-1] | s[i+(1<<(j-1))][j-1];
    }
}

int q;
cin >> q;

for(int i=0; i<q; i++){
    int L,R,target_idx;
    cin >> L >> R >> target_idx;

    int len = R-L+1;
    int j = log2(len);

    int left_block_start = L;
    int right_block_start = R - (1 << j) + 1;

    int range_or = s[left_block_start][j] | s[right_block_start][j];

    if(range_or == arr[target_idx]){
        cout <<"YES\n";
    }else{
        cout <<"NO\n";
    }
}

```

****Mr. Unascendable (Graph)**

```
void dfs(int r, int c, vector<vector<int>> &grid, vector<vector<bool>> &visited, int prevHeight)
{
    if (r < 0 || r >= m || c < 0 || c >= n)
        return;

    if (visited[r][c])
        return;

    if (grid[r][c] < prevHeight)
        return;

    visited[r][c] = true;

    dfs(r + 1, c, grid, visited, grid[r][c]);
    dfs(r - 1, c, grid, visited, grid[r][c]);
    dfs(r, c + 1, grid, visited, grid[r][c]);
    dfs(r, c - 1, grid, visited, grid[r][c]);
}

vector<vector<int>> grid(m, vector<int>(n));
for (int i = 0; i < m; i++)
{
    for (int j = 0; j < n; j++)
    {
        cin >> grid[i][j];
    }
}

vector<vector<bool>> reachedGreen(m, vector<bool>(n, false));
vector<vector<bool>> reachedRed(m, vector<bool>(n, false));

for (int i = 0; i < m; i++){
    dfs(i, 0, grid, reachedGreen, grid[i][0]);
}

for (int j = 0; j < n; j++){
    dfs(0, j, grid, reachedGreen, grid[0][j]);
}

for (int i = 0; i < m; i++){
    dfs(i, n - 1, grid, reachedRed, grid[i][n - 1]);
}

for (int j = 0; j < n; j++){
    dfs(m - 1, j, grid, reachedRed, grid[m - 1][j]);
}

for (int i = 0; i < m; i++){
    for (int j = 0; j < n; j++){
        if (reachedGreen[i][j] && reachedRed[i][j]){
            cout << "O ";
        }
        else{
            cout << "X ";
        }
    }
    cout << "\n";
}
```

****Westeros**

```
int V,E,Q;
while(!(cin >> V >> E >> Q)) return 0;

vector<string>cities(V);
for(int i=0; i<V; i++){
    cin >> cities[i];
}

map<string,vector<string>>Graph; //map stores which cities r directly connected to each other
for(int i=0; i<E; i++){
    string city1, city2;
    cin >> city1 >> city2;

    Graph[city1].push_back(city2); //since undirected add the cities both ways
    Graph[city2].push_back(city1);
}

for(int i=0; i<Q; i++){
    string src,des;
    int k;
    cin >> src >> des >>k;

    map<string,string>parent; //to remember from where we came from
    map<string,bool>visited; //cities we already visited
    queue<string>q; //cities waiting in line

    q.push(src);
    visited[src] = true;
    bool found = false;

    while(!q.empty()){
        string current = q.front();
        q.pop();

        if(current == des){
            found = true;
            break;
        }

        for(string neighbour : Graph[current]){
            if(!visited[neighbour]){
                visited[neighbour] = true;
                parent[neighbour] = current;
                q.push(neighbour);
            }
        }
    }

    if(!found){
        cout << "NO\n";
    }
    else{
        vector<string>path;
        string curr = des;
        while(curr != src){
            path.push_back(curr);
            curr = parent[curr];
        }
        path.push_back(src);
        reverse(path.begin(),path.end());
        int distance = path.size()-1;
        if(distance % k != 0){
            cout << "NO\n";
        }
        else{
            for(int j=0; j<path.size(); j+=k){
                cout << path[j] << (j + k < path.size() ? " " : "");
            }
            cout <<"\n";
        }
    }
}
```

**determine the exact number of islands in this archipelago.

```
void sinkIsland(int r, int c, vector<string> &grid)
{
    if (r < 0 || r >= m || c < 0 || c >= n)
        return;

    if (grid[r][c] == '0')
        return;

    grid[r][c] = '0';

    sinkIsland(r + 1, c, grid);
    sinkIsland(r - 1, c, grid);
    sinkIsland(r, c + 1, grid);
    sinkIsland(r, c - 1, grid);
}

if (!(cin >> m >> n))
    return 0;

vector<string> grid(m);

for (int i = 0; i < m; i++) {
    cin >> grid[i];
}

int islands = 0;

for (int i = 0; i < m; i++) {
    for (int j = 0; j < n; j++) {
        if (grid[i][j] == '1') {
            islands++;
            sinkIsland(i, j, grid);
        }
    }
}

cout << islands << endl;
return 0;
```

**Check if there exists a path between two nodes (Graph)

```

int n,m;

if(!(cin >> n >> m)) return 0;
vector<vector<int>>graph(n+1);

for(int i=0; i<m; i++){
    int u,v;
    cin >> u >> v;
    graph[u].push_back(v);
}

int src, des;
cin >> src >> des;

vector<bool>visited(n+1, false);
queue<int>q;

q.push(src);
visited[src] = true;
bool found = false;

while(!q.empty()){
    int curr = q.front();
    q.pop();

    if(curr == des){
        found = true;
        break;
    }

    for(int neighbour : graph[curr]){
        if(!visited[neighbour]){
            visited[neighbour] = true;
            q.push(neighbour);
        }
    }
}

```

to implement **Topological Sort algorithm to figure out who can bully him, and who he can bully in return.

```

int V,E,Q;
while(!(cin >> V >> E >> Q)) return 0;

vector<string>students(V);
map<string, vector<string>>>adj;
map<string, int>in_degree;

for(int i=0; i<V; i++){
    cin >> students[i];
    in_degree[students[i]] = 0;
}

for(int i=0; i<E; i++){
    string student1, student2;
    cin >> student1 >> student2;

    adj[student1].push_back(student2);
    in_degree[student2]++;
}

queue<string>q;
vector<string>topo_order;

for(int i=0; i<V; i++){
    if(in_degree[students[i]] == 0){
        q.push(students[i]);
    }
}

while(!q.empty()){
    string curr = q.front();
    q.pop();
    topo_order.push_back(curr);

    for(string neighbour : adj[curr]){
        in_degree[neighbour]--;

        if(in_degree[neighbour] == 0){
            q.push(neighbour);
        }
    }
}

```

```

if(topo_order.size() != V){
    cout <<"IMPOSSIBLE\n";
    return 0;
}

for(int i=0; i<V; i++){
    cout << topo_order[i] << (i == V-1 ? "" : " ");
}
cout << endl;

for(int i=0; i<Q; i++){
    string bully, victim;
    cin >> bully >> victim;

    queue<string>search_q;
    map<string,bool>visited;
    bool found = false;

    search_q.push(victim);
    visited[victim] = true;

    while(!search_q.empty()){
        string curr = search_q.front();
        search_q.pop();

        if(curr == bully){
            found = true;
            break;
        }

        for(string neighbour : adj[curr]){
            if(!visited[neighbour]){

                visited[neighbour] = true;
                search_q.push(neighbour);
            }
        }
    }
}

```

***if the graph is bipartite

```

int V;
if(!(cin >> V)) return 0;

vector<vector<int>>graph(V);
for(int i=0; i<V; i++){
    int n;
    cin >> n;

    for(int j=0; j<n;j++){
        int neighbour;
        cin >> neighbour;

        graph[i].push_back(neighbour);
    }
}

vector<int>color(V,-1); // -1 ->unpainted, 0->red, 1->blue
bool isBipartite = true;

```

```

,
vector<int> color(V, -1); // -1 ->unpainted, 0->red, 1->blue
bool isBipartite = true;

for(int start = 0; start < V; start++){
    if(color[start] == -1){
        queue<int> q;
        q.push(start);
        color[start] = 0;

        while(!q.empty()){
            int curr = q.front();
            q.pop();

            for(int neighbour : graph[curr]){
                if(color[neighbour] == -1){ //neighbour unpainted
                    color[neighbour] = 1-color[curr]; //paint it in opp color
                    q.push(neighbour);
                }
                else if(color[neighbour] == color[curr]){
                    isBipartite = false;
                    break;
                }
            }

            if(!isBipartite){
                break;
            }
        }

        if(!isBipartite){
            break;
        }
    }
}
}

```

***DSU IMpLeMentation

```

vector<int> parent;
vector<int> sz; // 'size' is a keyword in some contexts, 'sz' is safer
int components;
int max_sz;

// 1. Find with Path Compression
int find_set(int v) {
    if (v == parent[v])
        return v;
    return parent[v] = find_set(parent[v]);
}

// 2. Union by Size
void unite_sets(int a, int b) {
    a = find_set(a);
    b = find_set(b);

    if (a != b) {
        // Union by size: attach smaller to larger
        if (sz[a] < sz[b])
            swap(a, b);

        parent[b] = a;
        sz[a] += sz[b];

        // Update global tracking
        components--;
        max_sz = max(max_sz, sz[a]);
    }
}

```

```

int n, m;
if (!(cin >> n >> m)) return 0;

// Initialization
parent.resize(n + 1);
sz.resize(n + 1, 1); // Every node starts with size 1
iota(parent.begin(), parent.end(), 0); // parent[i] = i

components = n;
max_sz = 1;

for (int i = 0; i < m; i++) {
    int a, b;
    cin >> a >> b;

    unite_sets(a, b);

    // Output current number of disconnected groups and the largest group size
    cout << components << " " << max_sz << endl;
}

```

****Graph is k-Regular**

```

int V, E;
cin >> V >> E;

vector<int> degree(V + 1, 0);
while (E--)
{
    int u, v;
    cin >> u >> v;
    degree[u]++;
    degree[v]++;
}
int k = degree[1]; // first vertex
bool regular = true;

for (int i = 2; i <= V; i++)
{
    if (degree[i] != k)
    {
        regular = false;
        break;
    }
}

if (regular)
{
    cout << "Yes" << endl;
}
else
{
    cout << "No" << endl;
}
return 0;

```

****Implement Breadth First Search (BFS) for an undirected, unweighted graph.**

```

int V, E, s;
cin >> V >> E >> s;

vector<vector<int>> adj(V + 1); // 1-based indexing
for (int i = 0; i < E; i++)
{
    int u, v;
    cin >> u >> v;
    adj[u].push_back(v); //for undirected graph, u to v, v to u
    adj[v].push_back(u);
}

// adjacency list

cout << "Adjacency list:" << endl;
for (int i = 1; i <= V; i++)
{
    cout << i << ": ";
    for (int v : adj[i])
        cout << v << " ";
    cout << endl;
}

// BFS attributes

vector<string> color(V + 1, "WHITE"); // white -> unvisited
vector<int> distance(V + 1, -1); // distance from source
vector<int> predecessor(V + 1, -1); // to find parent

queue<int> q;
q.push(s);
color[s] = "GRAY"; //since source has been pushed, hence its processing
distance[s] = 0; // distance from itself i.e distance a from a is 0

vector<int> bfs_order;
vector<pair<int, int>> bfs_edges;

while (!q.empty())
{
    int u = q.front(); //taking vertex u from q
    q.pop(); // to push it to another place, we need to pop from q first
    bfs_order.push_back(u); // adding to bfs_order

    for (int v : adj[u]) // checking the neighbour v of u
    {
        if (color[v] == "WHITE") // if not visited
        {
            color[v] = "GRAY"; // then visiting
            distance[v] = distance[u] + 1;
            predecessor[v] = u;
            q.push(v);
            bfs_edges.push_back({u, v}); // making it part of
        }
    }
    color[u] = "BLACK";
}

// BFS order

cout << "\nBFS order: ";
for (int u : bfs_order) // shows the order of vertices being discovered
    cout << u << " ";
cout << endl;

// Distances

cout << "\nDistance from source: " << endl;
for (int i = 1; i <= V; i++)
{
    cout << i << "(" << distance[i] << ") ";
}
cout << endl;

```

```

// Paths

cout << "\nPaths from source: " << endl;
for (int i = 1; i <= V; i++)
{
    if (i == s)
    {
        cout << i << ": " << i << endl;
        continue;
    }
    if (distance[i] == -1){
        continue;
    }
    vector<int> path;
    int x = i; // to follow predecessor up to source
    while (x != -1)
    {
        path.push_back(x); //collecting vertices in path order
        x = predecessor[x];
    }
    reverse(path.begin(), path.end()); //to get order from s to i
    cout << i << ": ";
    for (int j = 0; j < path.size(); j++)
    {
        cout << path[j];
        if (j != path.size() - 1)
            cout << "->";
    }
    cout << endl;
}

```

// BFS tree edges

```

cout << "\nEdges of BFS tree:" << endl;
for (auto e : bfs_edges)
    cout << e.first << " " << e.second << endl;

```

***Implement Depth First Search (DFS) for a directed unweighted graph.

```

int time_count = 0;
vector<string> color;
vector<int> discovery;
vector<int> finish_time;
vector<int> predecessor;
vector<int> dfs_order;

void dfs_visit(int u, vector<vector<int>>&adj){
    time_count = time_count+1;
    discovery[u] = time_count; //marking the curr time when its discovered
    color[u] = "GRAY";
    dfs_order.push_back(u); //listing order of discovering

    for(int v : adj[u]){
        if(color[v] == "WHITE"){
            predecessor[v] = u;
            dfs_visit(v,adj); //going to the dept of v
        }
    }
    color[u] = "BLACK";
    time_count = time_count+1;
    finish_time[u] = time_count;
}

```

```

int V, E;
cin >> V >> E;

color.assign(V + 1, "WHITE");
discovery.assign(V + 1, 0);
finish_time.assign(V + 1, 0);
predecessor.assign(V + 1, -1);

vector<vector<int>>>adj(V+1);
vector<pair<int,int>>>edges;

for(int i=1; i<=E; i++){
    int u,v;
    cin >> u >> v;
    adj[u].push_back(v);
    edges.push_back({u,v});
}

for(int i=1; i<=V; i++){
    cout << i << ": ";
    for(int v: adj[i]){
        cout << v << " ";
    }
    cout << endl;
}

for(int i =1; i<=V; i++){
    if(color[i] == "WHITE"){
        dfs_visit(i,adj);
    }
}
cout << "DFS order: ";
for(int i =0; i<dfs_order.size(); i++){
    cout << dfs_order[i] << " ";
}
cout << endl;

cout << "Timestamps of Vertex(discovery/finishing): " << endl;
for(int i =1; i<=V; i++){
    cout << i << "(" << discovery[i] << "/" << finish_time[i] << ")" << endl;
}

cout << "\nEdge classification:" << endl;
for(auto e : edges){
    int u = e.first;
    int v = e.second;

    cout << u << " -> " << v << " : ";

    if (predecessor[v] == u){
        cout << "Tree Edge";
    }
    else if (discovery[v] < discovery[u] && finish_time[u] < finish_time[v]) //v opened by u and u closed by v{
        cout << "Back Edge";
    }
    else if (discovery[u] < discovery[v] && finish_time[v] < finish_time[u]) // u-> v but not as a direct predecessor{
        cout << "Forward Edge";
    }
    else{
        cout << "Cross Edge";
    }
    cout << endl;
}

```

****Union by rank and path compression**

```

class Basic_DSU
{
private:
    vector<int> parent;
    vector<int> rank;

public:
    Basic_DSU(int size)
    {
        parent.resize(size);
        rank.resize(size, 0);
        for (int i = 0; i < size; i++)
        {
            parent[i] = i;
        }

        int find(int i)
        {
            if (parent[i] == i)
            {
                return i;
            }
            return find(parent[i]);
        }

        void union_set(int i, int j)
        {
            int irep = find(i);
            int jrep = find(j);
            if (rank[irep] > rank[jrep]) {
                parent[jrep] = irep; // Attach shorter tree under taller tree
            }
            else if (rank[irep] < rank[jrep]) {
                parent[irep] = jrep;
            }
            else { // Attaching one under another increases height by 1
                parent[irep] = jrep;
                rank[jrep] ++;
            }
        }

        void print()
        {
            for (size_t i = 0; i < parent.size(); i++)
            {
                cout << parent[i] << '(' << rank[i] << ')' << " ";
            }
            cout << endl;
        }
    }
}

```

****Represent Graph using Adjacency List & Adjacency Matrix**

```

int V, E;
cin >> V >> E;

vector<vector<int>> adjList(V + 1); // adjList[i] stores all neighbors of vertex i
// Size is V+1 → so we can use 1-based indexing

vector<vector<int>> adjMatrix(V + 1, vector<int>(V + 1, 0)); // 2d matrix

for (int i = 0; i < E; i++)
{
    int u, v;
    cin >> u >> v;

    adjList[u].push_back(v);
    adjList[v].push_back(u);

    adjMatrix[u][v] = 1; // Same edge stored in matrix form
    adjMatrix[v][u] = 1;
}
cout << "Adjacency List:" << endl;
for (int i = 1; i <= V; i++)
{
    cout << i << ":";
    for (int j = 0; j < adjList[i].size(); j++)
    {
        cout << adjList[i][j] << " ";
    }
    cout << endl;
}

cout << endl;

cout << "Adjacency Matrix:" << endl;
for (int i = 1; i <= V; i++)
{
    for (int j = 1; j <= V; j++)
    {
        cout << adjMatrix[i][j] << " ";
    }
    cout << endl;
}

```

***: Topological Sort Using Queue Approach

```

int N, M;
cin >> N >> M;

vector<vector<int>> adj(N);
vector<int> indegree(N, 0);

for (int i = 0; i < M; i++) {
    int a, b;
    cin >> a >> b;
    adj[a].push_back(b);
    indegree[b]++;
}

queue<int> q;
vector<int> topo;

for (int i = 0; i < N; i++) {
    if (indegree[i] == 0)
        q.push(i);
}

while (!q.empty()) {
    int u = q.front();
    q.pop();
    topo.push_back(u);

    for (int v : adj[u]) {
        indegree[v]--;
        if (indegree[v] == 0)
            q.push(v);
    }
}

```

****Implementing a Basic Hash Table with Collision Resolution Strategy**

```

int hashFunction(int key, int tablesizel){
    return key%tablesizel;
}

// f(i)=i*hash2(x); hash2(x) = R - (x % R) with R=7
int hash2(int x){
    int R = 7;
    return R-(x%R);
}

int main(){

    int choice ,N,Q;
    cin >> choice >> N >> Q;

    vector<int>hashTable(N,-1);
    int insertedCount = 0;

    for(int q=0; q<Q; q++){
        int key;
        cin >> key;

        bool inserted = false;
        int attempt = 0;

        while(attempt < 6 && !inserted){
            int index;
            int h = hashFunction(key,N);

            if(choice == 1){
                //Linear prob
                index = (h+attempt)%N;
            }
            else if(choice == 2){
                //quadratic prob
                index = (h + attempt*attempt)%N;
            }

            else if(choice == 3){
                //double hashing
                index = (h+attempt*hash2(key))%N;
            }

            if(hashTable[index] == -1){
                hashTable[index] = key;
                insertedCount++;
                cout << "Inserted : Index-" << index << "(L.F=" << (double)insertedCount/N << ")" << endl;
                inserted = true;
            }
            else{
                cout << "Collision: Index-" << index << endl;
            }
            attempt++;
        }

        if(!inserted){
            cout << "Input Abandoned" << endl;
        }
    }
}

```

****the pairs of integers whose summation is equal to 'target'**

```

nt main() {
    vector<int> num;
    int n;
    int target;
    while (cin >> n && n != -1) {
        num.push_back(n);
    }
    cin >> target;

    unordered_map<int, int> freq;
    vector<pair<int, int>> result;

    for(int x : num) {
        freq[x]++;
    }

    for(int x: num) {
        int complement = target - x;
        if(freq[x] > 0 && freq[complement] > 0) {
            if(x == complement && freq[x] > 2) {
                continue;
            }
            result.push_back({x, complement});
            freq[x]--;
            freq[complement]--;
        }
    }
    if(result.empty()) {
        cout << "No pairs found" << endl;
    } else {
        for(size_t i=0; i< result.size(); i++) {
            cout << "(" << result[i].first << ", " << result[i].second << ")";
            if(i < result.size()-1) {
                cout << ", ";
            }
        }
    }
}

```

****Basic operations of the Trie data structure**

```

struct TrieNode {
    bool endmark;
    TrieNode* next[26];

    TrieNode() {
        endmark = false;
        for(int i=0; i<26; i++) {
            next[i] = nullptr;
        }
    }
} * root;

void insert(string str) {
    TrieNode* curr = root;

    int len = str.length();
    for(int i=0; i<len; i++) {
        int idx = str[i] - 'a';

        if(curr->next[idx] == nullptr) {
            curr->next[idx] = new TrieNode();
        }
        curr = curr->next[idx];
    }
    curr->endmark = 1;
}

```

```

bool search(string str){
    TrieNode* curr = root;
    int len = str.length();
    for(int i=0; i<len; i++){
        int idx = str[i] - 'a';
        if(curr->next[idx] == nullptr){
            return false;
        }
        curr = curr->next[idx];
    }
    return curr->endmark;
}

void displayrecursion(TrieNode *curr, string &str){
    if(curr->endmark){
        cout << str << endl;
    }
    for(int i=0; i<26; i++){
        if(curr->next[i] != nullptr){
            char ch = i + 'a';
            str.push_back(ch);
            displayrecursion(curr->next[i], str);
            str.pop_back();
        }
    }
}

void display(){
    string str = "";
    displayrecursion(root, str);
}

```

```

int main(){
    root = new TrieNode();

    string line;
    getline(cin, line);

    stringstream ss(line);
    string word;

    while(ss >> word){
        insert(word);
    }

    display();
    cout << endl;

    getline(cin, line);
    stringstream ss2(line);

    while(ss2 >> word){
        if(search(word)){
            cout << "T" << " ";
        }
        else{
            cout << "F" << " ";
        }
    }
    return 0;
}

```

**a set of words is stored in a Dictionary. Given a *prefix*, your task is to find out how many words start with it.

Struct TrieNode()

```

void toLowercase(string &str)
{
    for (int i = 0; i < str.length(); i++)
    {
        if (str[i] >= 'A' && str[i] <= 'Z')
        {
            str[i] = str[i] - 'A' + 'a';
        }
    }
}

bool search(string str)
{
    TrieNode *curr = root;
    int len = str.length();
    for (int i = 0; i < len; i++)
    {
        int idx = str[i] - 'a';
        if (curr->next[idx] == nullptr)
        {
            return false;
        }
        curr = curr->next[idx];
    }
    return curr->endmark;
}

```

```

void insert(string str)
{
    TrieNode *curr = root;
    if(search(str)) return;

    int len = str.length();
    for (int i = 0; i < len; i++)
    {
        int idx = str[i] - 'a';

        if (curr->next[idx] == nullptr)
        {
            curr->next[idx] = new TrieNode();
        }
        curr = curr->next[idx];
        curr->count++;
    }
    curr->endmark = 1;
}

int prefixCount(string& str){
    TrieNode* curr = root;
    for(int i=0; i<str.length(); i++){
        int idx = str[i] - 'a';
        if(curr->next[idx] == nullptr){
            return 0;
        }
        curr = curr->next[idx];
    }
    return curr->count;
}

```

```

void DeleteTri(TrieNode* curr) {
    if(curr == nullptr) return;

    for(int i=0; i<26; i++){
        if(curr->next[i]){
            DeleteTri(curr->next[i]);
        }
        delete curr;
    }
}

int main() {
    root = new TrieNode();
    int n,q;
    cin >> n >> q;

    string word;
    for(int i=0; i<n; i++){
        cin >> word;
        toLowercase(word);
        insert(word);
    }

    for(int i=0; i<q; i++){
        cin >> word;
        toLowercase(word);
        cout<< prefixCount(word) << endl;
    }
    DeleteTri(root);
    return 0;
}

```

****at most three** products after each character of searchWord is typed.

Struct TrieNode()

Void insert(string str)

```

void dfs(TrieNode *curr, string wordsofar, vector<string>&suggestion){
    if(suggestion.size() == 3){
        return;
    }

    if(curr->endmark){
        suggestion.push_back(wordsofar);
    }

    for(int i = 0; i < 26; i++){
        if(curr->next[i] != nullptr){
            char ch = i + 'a';
            dfs(curr->next[i], wordsofar+ch, suggestion);
            if(suggestion.size() == 3) return;
        }
    }
}

void getSuggestions(string searchword){
    TrieNode* curr = root;
    bool pathbroken = false;
    string prefix = "";

    for(char c : searchword){
        c = tolower(c);
        prefix += c;
        int idx = c - 'a';

        if(pathbroken || curr->next[idx] == nullptr){
            pathbroken = true;
            cout << "Null" << endl;
        }
        else{
            curr = curr->next[idx];
            vector<string>suggestion;
            dfs(curr, prefix, suggestion);

            else{
                curr = curr->next[idx];
                vector<string>suggestion;
                dfs(curr, prefix, suggestion);

                for(const string &s : suggestion){
                    cout << s << " ";
                }
                cout << endl;
            }
        }
    }
}

int main(){
    root = new TrieNode();
    string line, word;
    getline(cin, line);

    stringstream ss(line);
    while(ss >> word){
        insert(word);
    }

    string searchword;
    cin >> searchword;
    getSuggestions(searchword);
}

```

****maximum XOR value between any two numbers in the collection**

```

struct TrieNode{
    TrieNode* left;    // bit 0
    TrieNode* right;   //bit 1

    TrieNode(){
        left = nullptr;
        right = nullptr;
    }

    ~TrieNode(){
        if(left) delete left;
        if(right) delete right;
    }
};

class Solution{
    TrieNode* root;

public:
    Solution(){
        root = new TrieNode;
    }

    ~Solution(){
        delete root;
    }

    void insert(int num){
        TrieNode* curr = root;
        for(int i = 31; i >= 0; i--){
            int bits = (num >> i) & 1;
            if(bits == 0){
                if(curr->left == nullptr){

```

```

        if(bits == 0){
            if(curr->left == nullptr){
                curr->left = new TrieNode();
            }
            curr = curr->left;
        }else{
            if(curr->right == nullptr){
                curr->right = new TrieNode();
            }
            curr = curr->right;
        }
    }
}

int findMaxXorNum(int num){
    TrieNode* curr = root;
    int currXor = 0;

    for(int i=31; i>=0; i--){
        int bit = (num >> i) & 1;

        int desireNum = 1-bit;

        if(desireNum == 0){
            if(curr->left != nullptr){
                currXor += (1<<i);
                curr = curr->left;
            }else{
                curr = curr->right;
            }
        }else{
            if(curr->right != nullptr){
                currXor += (1 << i);
                curr = curr->right;
            }else{
                curr = curr->left;
            }
        }
    }
    return currXor;
}

int solve(vector<int>& num){
    if(num.size() < 2) return 0;

    for(int x : num){
        insert(x);
    }
    int globalMax = 0;
    for(int x : num){
        globalMax = max(globalMax, findMaxXorNum(x));
    }
    return globalMax;
}

};

int main(){
    string line;
    getline(cin, line);

    stringstream ss(line);
    int num;
    vector<int> nums;
    while (ss >> num){
        nums.push_back(num);
    }
    Solution solver;
    cout << solver.solve(nums) << endl;

    return 0;
}

```

** the abbreviation fits the words in that list

```
bool matching(string word, string abbre){
    int wordptr = 0;
    int abbreptr = 0;

    while (wordptr < word.length() && abbreptr < abbre.length()){
        if (word[wordptr] == abbre[abbreptr]){
            abbreptr++;
            wordptr++;
        }
        else if (isupper(word[wordptr])){
            return false;
        }
        else{
            wordptr++;
        }
    }

    if (abbreptr < abbre.length()){
        return false;
    }

    while (wordptr < word.length()){
        if (isupper(word[wordptr])){
            return false;
        }
        wordptr++;
    }
    return true;
}

int main()
{
    vector<string> words;
    string input;

    while (cin >> input && input != "-1"){
        words.push_back(input);
    }

    string abbreviation;
    cin >> abbreviation;

    for (const string &s : words){
        if (matching(s, abbreviation)){
            cout << "T" << " ";
        }
        else{
            cout << "F" << " ";
        }
    }
    cout << endl;
    return 0;
}
```

```
#include<sstream>
using namespace std;

void solution(){
    string line;
    while(getline(cin, line)){

        for(int i=0; i<line.length(); i++){
            if(ispunct(line[i])){
                line[i] = ' ';
            }
        }
        stringstream ss(line);    //extracting the words one by one
        string word;

        map<string, int>freq;
        while(ss >> word){
            freq[word]++;
        }

        for(auto pair : freq){

            string words = pair.first;
            int count = pair.second;
            if(count > 1){
                cout << words << " " << count << endl;
            }
        }
    }
}

int main(){
    solution();
    return 0;
}
```

[illegible]

** to print the words that consist of the same **unique character set**.

```
]string getHash(string word) {
    set<char>uniquechar;

    for(char c: word){
        uniquechar.insert(c);
    }

    string signature = "";
    for(char c : uniquechar){
        signature += c;
    }
    return signature;
}

]void solve() {

    string sentence, target;
    if(!getline(cin,sentence)) return;
    if(!(cin >> target)) return;

    string targetHash = getHash(target);
    stringstream ss(sentence);

    string currword;
    while(ss >> currword){
        if(getHash(currword) == targetHash){
            cout << currword << " ";
        }
    }
    cout << endl;
}

]int main() {

    solve();
    return 0;
}
```